

IDENTIFYING BOTTLENECKS AND INTEGRATING LEARNED OPTIMIZATIONS INTO RELATIONAL DATABASES

Internal Supervisor: Dr. Buddhika Karunaratne
External Supervisor: Dr. Srinath Perera

Team Members:

Madara Mendis
Madushan Suriyanbandara
Nisith Dissanayake
Nithika Dias

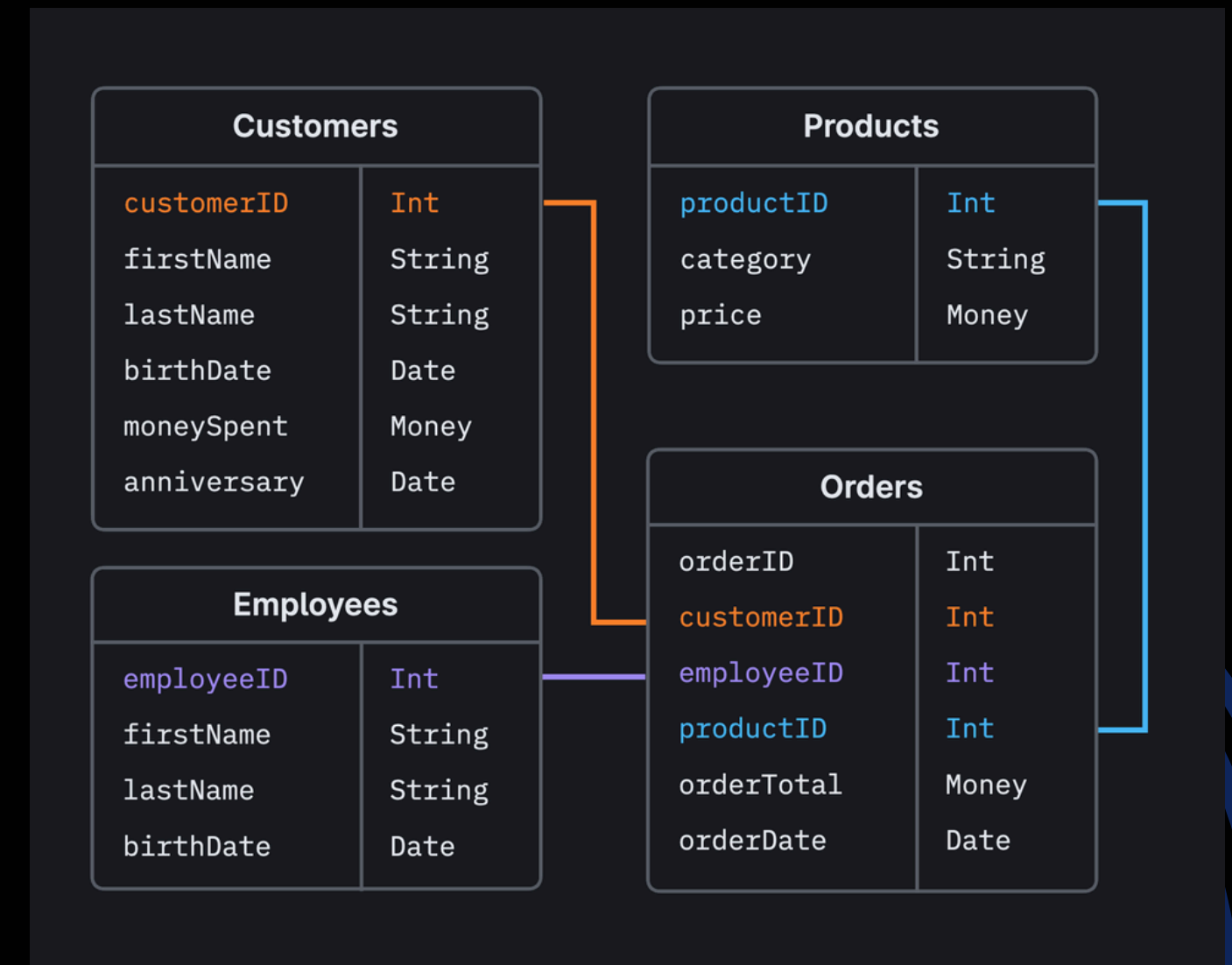
AGENDA

Introduction	3
Problem Statement	7
Motivation	12
Research Objectives	15
Literature Review	7
Proposed Methodology	37
Feasibility & Scope	42
Deliverables	45
Timeline & Work Distribution	47
References	49

Introduction

What is a Relational Database?

- A relational database is a collection of information that organizes data in predefined relationships. [1]
- Where data is stored in tables of columns and rows. making it easy to see and understand how different data structures relate to each other.



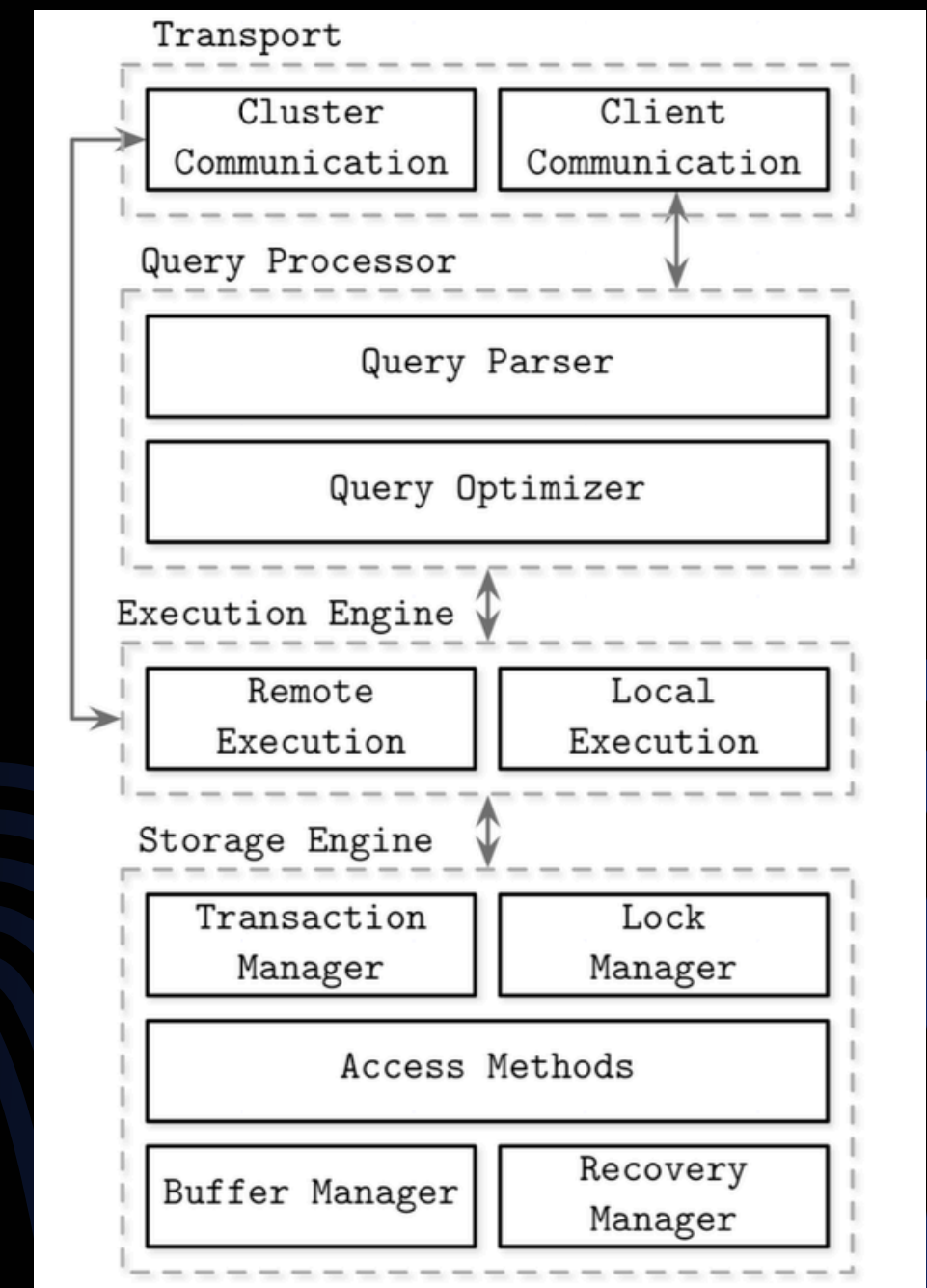
[1] <https://cloud.google.com/learn/what-is-a-relational-database?hl=en>

Why are RDMSs Important

- Relational Database are the foundation of most modern applications
- These include applications from banking, healthcare, transport systems to e-commerce.
- They handle large-scale structured data while ensuring data integrity is maintained and adequate execution time is maintained.
- Examples include: PostgreSQL, MySQL, OracleDB etc.

High Level Internal Design of a RDMS

- Query Processor: parses the query and determines an optimal plan to execute the query
- Execution Engine: runs the execution plan chosen by the optimizer.
- Storage Engine: manages data files, indexes, buffer cache



Problem Statement

Problem Context

- Techniques to execute queries and configure databases and other features have generally remained static over the years.
- But recent studies have showed that for complex queries in large databases there is room for improvement.
- Eg : *“We investigate the quality of industrial-strength cardinality estimators and find that all estimators routinely produce large errors.” Leis et al (2015)*
- This has created a shift into introducing “Learned Optimization Techniques” for databases

Problem Context

Many Learned Optimization techniques have been published and introduced under different categories of bottlenecks, but most of them have been evaluated in isolation.

ALECE: An Attention-based Learned Cardinality Estimator for SPJ Queries on Dynamic Workloads (Extended)

Pengfei Li
Alibaba Group, China
lpf367135@alibaba-inc.com

Wenqing Wei
Alibaba Group, China
weiwenqing.wwwq@alibaba-inc.com

Rong Zhu †
Alibaba Group, China
red.zr@alibaba-inc.com

Neo: A Learned Query Optimizer

Ryan Marcus¹, Parimarjan Negi², Hongzi Mao², Chi Zhang¹,
Mohammad Alizadeh², Tim Kraska², Olga Papaemmanouil¹, Nesime Tatbul^{2,3}
¹Brandeis University ²MIT ³Intel Labs

Lu †
sity, Denmark
ruc.dk

asets are often complex
owerful enough to build
maries and SQL query
ain join predicates but
i particular summaries

Learning to Optimize Join Queries With Deep Reinforcement Learning

Sanjay Krishnan^{1,2}, Zongheng Yang¹, Ken Goldberg¹, Joseph M. Hellerstein¹, Ion Stoica¹
¹RISELab, UC Berkeley ²Computer Science, University of Chicago
skr@cs.uchicago.edu {zongheng, goldberg, hellerstein, istoica}@berkeley.edu

DOI: 10.1145/nnnnnnnn.nnnnnnnn

ABSTRACT
Query optimization
in database system
past decades, quer
components that
specific workloads
coming and insiro

Apr 2019

ABSTRACT

PRICE: A Pretrained Model for Cross-Database Cardinality Estimation

Tianjing Zeng¹, Junwei Lan^{1,2,#}, Jiahong Ma^{1,3,#}, Wenqing Wei^{1,2}, Rong Zhu^{1,*}, Pengfei Li¹, Bolin Ding^{1,*}, Defu Lian², Zhewei Wei³, Jingren Zhou^{1,*}
¹Alibaba Group, ²University of Science and Technology of China, ³Renmin University of China

ABSTRACT
Cardinality estimation (CardEst) is essential for optimizing query execution plans. Recent ML-based CardEst methods achieve high accuracy but face deployment challenges due to high preparation costs and lack of transferability across databases. In this paper, we propose PRICE, a Pretrained multi-table CardEst model, which addresses these limitations. PRICE takes low-level but transferable features w.r.t. data distributions and query information and elegantly applies self-attention models to learn meta-knowledge to compute cardinality in any database. It is generally applicable to any unseen new database to attain high estimation accuracy, while its preparation cost is as little as the basic one-dimensional histogram-based CardEst methods. Moreover, PRICE can be finetuned to further enhance its performance on any specific database. We pretrained PRICE using 30 diverse datasets, completing the process in about 5 hours with a resulting model size of only about 40MB. Evaluations show that PRICE consistently outperforms existing methods, achieving the highest estimation accuracy on several unseen databases and generating faster execution plans with lower overhead. After finetuning with a small volume of database-specific queries, PRICE could even find plans very close to the optimal ones. Meanwhile, PRICE is generally applicable to different settings such as data updates, data scaling, and query workload shifts. We have made all of our data and codes publicly available at <https://github.com/StCarmen/PRICE>.

1 INTRODUCTION
Cardinality estimation (CardEst), which predicts the number of tuples of each SQL query before execution, plays a crucial role in query optimization in DBMS. Accurate CardEst could significantly help to identify highly efficient execution plans, thus enhancing the performance of DBMS. As a result, CardEst methods have been extensively studied in the literature, but now their performance is still far from satisfactory.

Background and Challenges. Traditional methods, including one-dimensional (1-D) histograms [63, 65] and sampling [31, 44],

or build models mapping featurized queries to their cardinalities using collected query workload (query-driven methods [21, 22]) or both (hybrid methods [46]). Although existing benchmarks and evaluations [28, 38] have verified that these ML-based methods could attain high accuracy to find better execution plans, their explicit shortcomings still prevent them from being truly applicable: 1) These methods are all data-specific and not transferable across different databases. Data-driven approaches model the specific underlying data distribution for each dataset. Query-driven approaches typically represent features (e.g., the values of attributes or table names) as hard encoded vectors, which can not be resolved in other datasets having different tables and attributes. 2) The preparation cost of existing ML-based methods is much higher than traditional CardEst methods. For each new database, they are required to collect training data, i.e., the sampled tuples in data-driven methods and the executed query workload in query-driven methods, tune hyper-parameters, train the models, and maintain the models for inference, which is time and space-consuming. 3) The performance of these ML-based methods is not stable [28, 38]. Data-driven methods often have prior assumptions on data distributions (e.g., DeepDB [35] assumes attributes are not highly correlated), while query-driven models can not perform well on queries having distinct distributions with the training workload [86]. Some methods [28, 38] can not well adapt to dynamic settings. We summarize the main properties of the above CardEst methods in Table 1. What we desire is a CardEst method that could be effortlessly deployed on any unseen new database with minimal preparations (akin to traditional methods) while still maintaining high but stable accuracy comparable to ML-based methods. **Motivations and Our Contributions.** We note that there has been some exploration into developing general models for cost estimation [14, 32–34] or single-table CardEst [52], but our goal is to resolve the most practical, but also complex, CardEst on multi-table join queries. We draw some inspiration from the success of pretrained models in the NLP domain, such as BERT [19] and ChatGPT [15]. They organize information hierarchically from the basic

ION
s been studied for more than four
ntinues to be an active area of re-
e problem’s combinatorial complexity
is use of *heuristics*. For example, clas-
s dynamic programs often restrict their
i shapes (e.g., “left-deep” plans). Query
i apply further heuristics to large join
[4] or randomized [40] algorithms. In-
istics can break down (by definition),
plans [29].
dvances in machine learning, a new
arch explores replacing programmed
d ones [11, 25, 26, 32–34, 37, 41]. In-
i, this paper explores the natural ques-
dataset-specific join search strategies
ing a given cost model and plan space,
earch over all possible join plans for a
: hope is to learn tailored search strate-
s of previous planning instances that
arch time for future planning.
hat join ordering has a deep algorithm-
reinforcement Learning (RL) [47]. Join
structure is the same problem structure
e exploit this algorithmic connection
to a traditional query optimizer; any
algorithm is used, a policy learned
can just as easily be applied. This in-
hieve two key benefits. First, we can
ur solution into many optimizers with
architecture. Second, we exploit the

arXiv:2406.01027v1 [cs.DB] 3 Jun 2024

Problem Context

Most Learned Optimization techniques have also not been tested within a database to verify performance and implementation feasibility.



Core Problem

Although many learned optimization techniques have been proposed, a combination of techniques, addressing bottlenecks in different areas, have not yet been directly integrated to a database and evaluated.

Motivation

Why This Research is Important?

- Databases are the backbone of data-driven applications.
- Query performance depends on:
 - Query optimizer
 - Configuration “knobs”
 - Database internals like Indexes
- Weakness in any one area → can cause severe slowdowns.
- Studies show reducing these errors leads to large performance gains.

Research Rationale & Gaps

- Growing interest in AI-driven databases (learned optimizers, tuning, design).
- Current research focuses on isolated techniques rather than system integration.
- Need for:
 - Realistic evaluations & benchmarks.
 - Lifecycle management (retraining, safe fallback).
 - Proper integration testing and ensuring feasibility for practitioners.



Research Objectives

Research Objectives



- To identify and analyze bottlenecks of current database systems, where improvements would lead to significant performance improvements.
- To identify and analyze the existing techniques that address these performance bottlenecks.
- To implement these techniques directly within database systems.
- Explore the potential to improve the techniques.
- Publish the findings of this research.

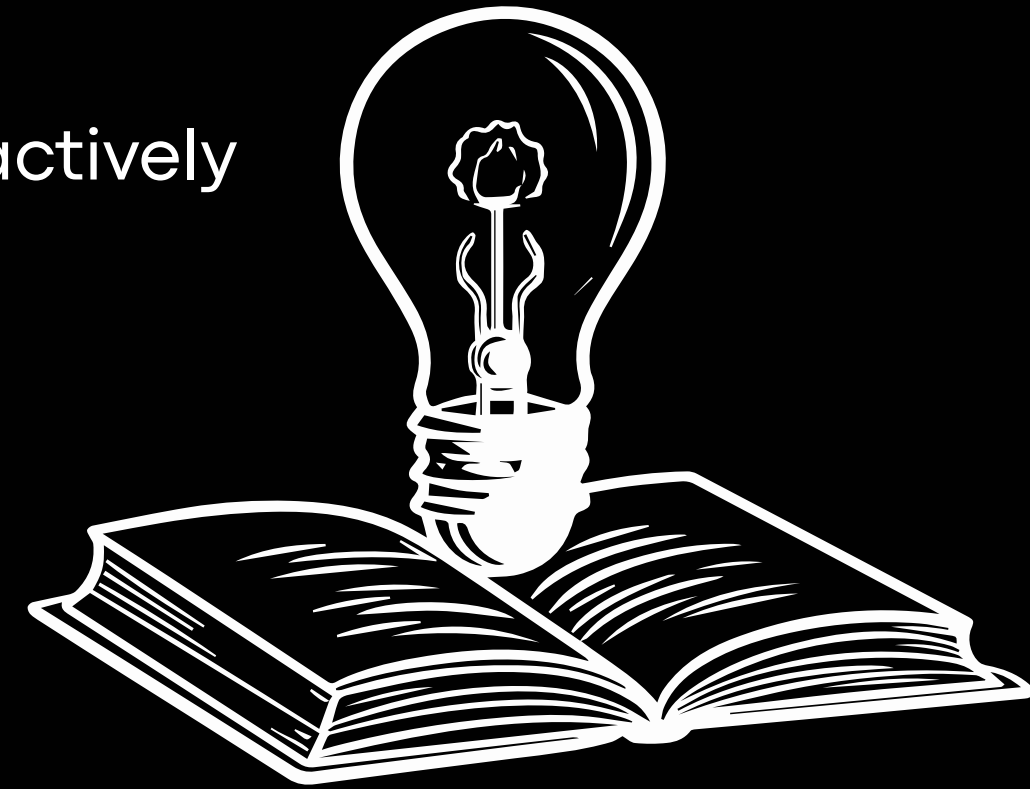
Literature Review

What did we focus on ?

In our literature review we have mainly focused on exploring key bottlenecks within relational databases and identifying emerging ML techniques to overcome these bottlenecks.

We identified three major bottlenecks that the database community is actively researching to optimize with Machine Learning concepts which are,

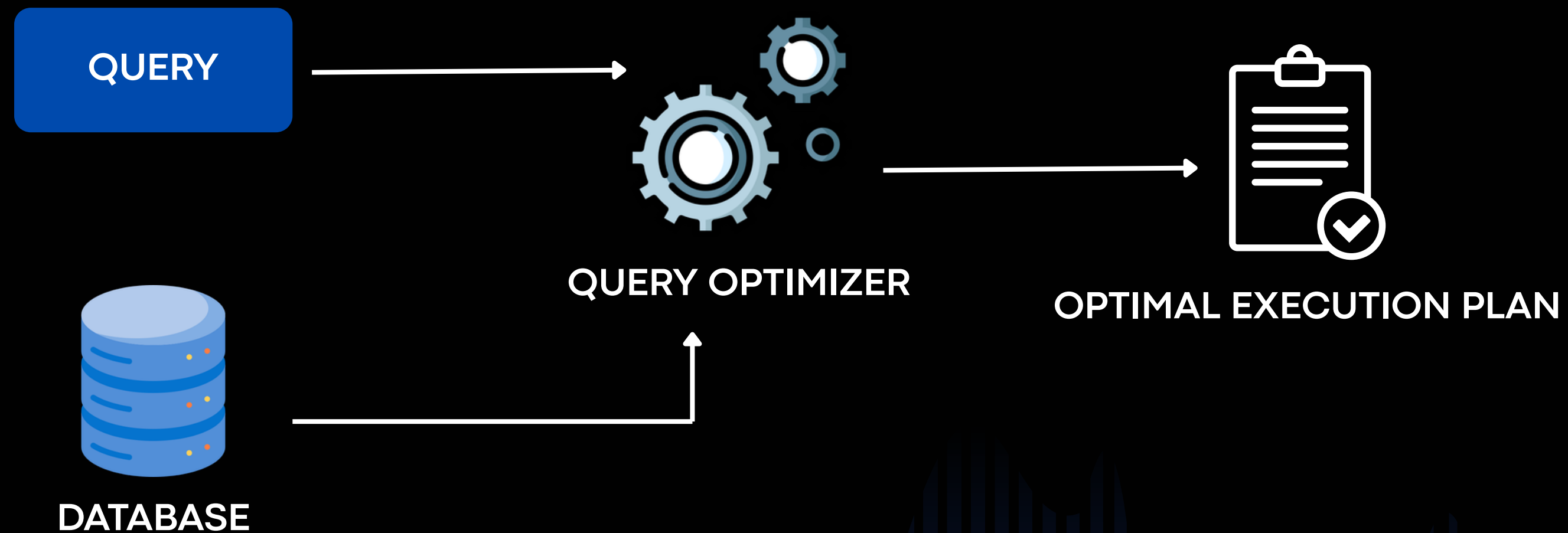
1. Query Optimization
2. Knob Tuning
3. Indexing



Query Optimization

Query Optimizer

The responsibility of Query Optimizer in a database is determining the most efficient way to execute a given SQL query.



QUERY OPTIMIZATION BOTTLENECKS

There are three main components in a Query Optimizer.

CARDINALITY ESTIMATION

Predicts the number of tuples produced by each intermediate operator in a query plan. Relies on statistical metadata and assumptions about data distribution.

COST MODEL

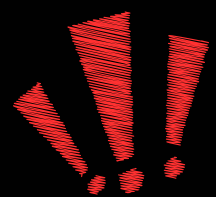
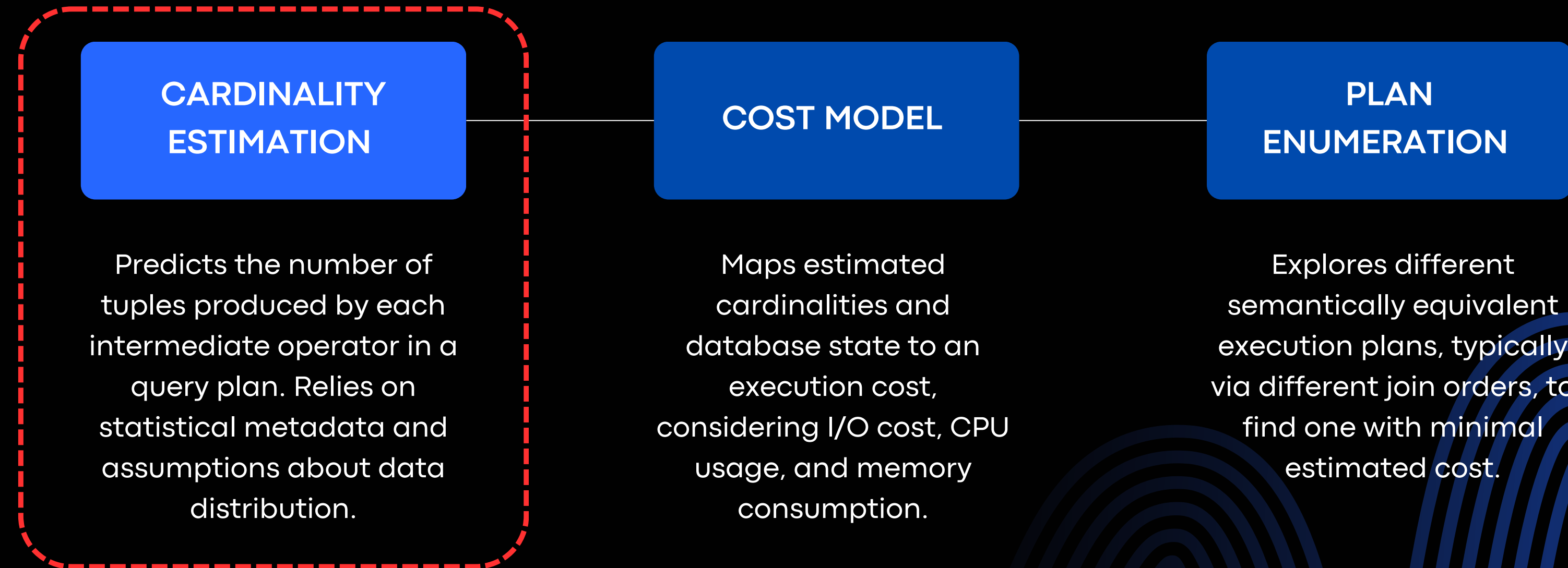
Maps estimated cardinalities and database state to an execution cost, considering I/O cost, CPU usage, and memory consumption.

PLAN ENUMERATION

Explores different semantically equivalent execution plans, typically via different join orders, to find one with minimal estimated cost.

“The root of all evil, the Achilles Heel of query optimization, is the estimation of the size of intermediate results, known as cardinalities. Everything in cost estimation depends upon how many rows will be processed, so the entire cost model is predicated upon the cardinality model.”

Is Query Optimization a “Solved” Problem? by Guy Lohman

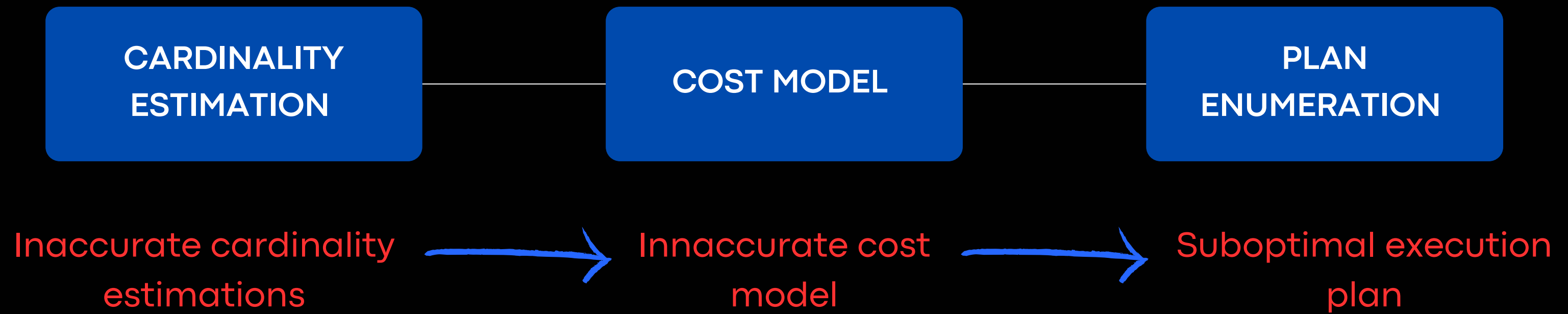


KEY

BOTTLENECK

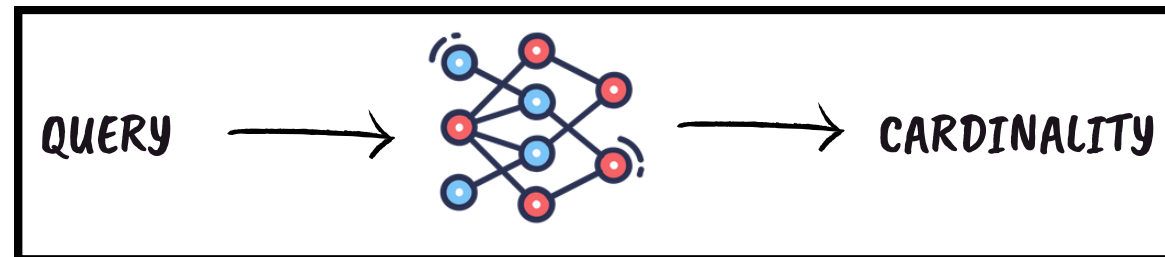
EXISTING CARDINALITY ESTIMATION METHODS FREQUENTLY PRODUCE INACCURATE ESTIMATES FOR INTERMEDIATE RESULTS, ESPECIALLY IN THE PRESENCE OF CORRELATED ATTRIBUTES, SKEWED DISTRIBUTIONS, OR COMPLEX PREDICATES.

Inaccurate cardinality estimations leads to suboptimal execution plans



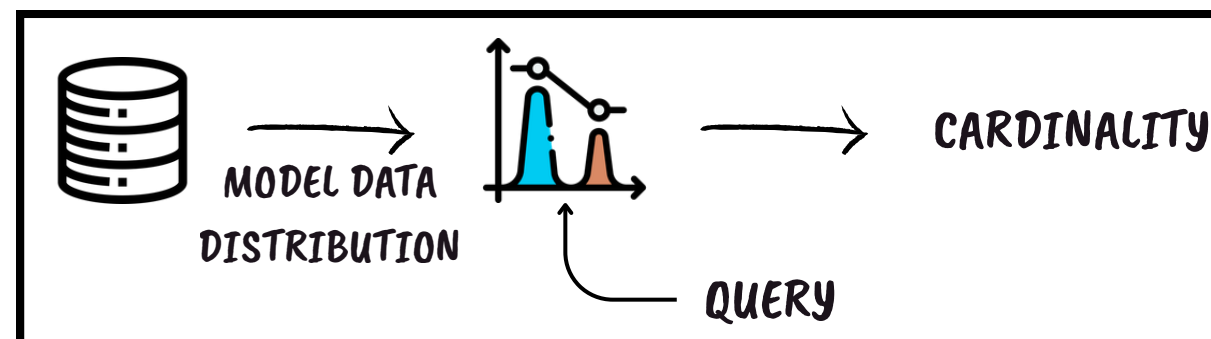
LEARNED CARDINALITY ESTIMATION

QUERY DRIVEN



LEARN MAPPING FROM,
QUERY FEATURES → CARDINALITY (SUPERVISED)
EXAMPLE: MSCN[6], ALECE[1]

DATA DRIVEN



MODEL DATA DISTRIBUTIONS (UNSUPERVISED/GENERATIVE)
EXAMPLE: DEEPDB[2], NEUROCARD[3]

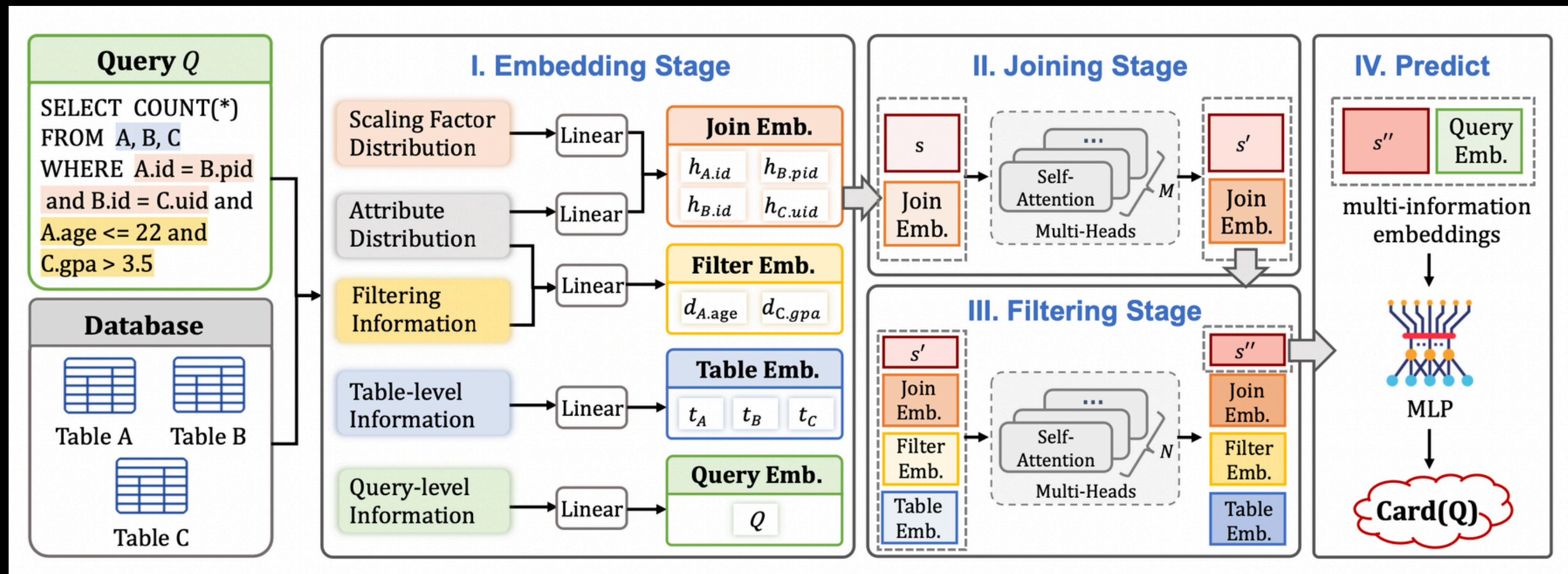
HYBRID MODELS

QUERY FEATURES + DATA SUMMARIES
EXAMPLE: ALECE (HYBRID) [1]

LEARNED CARDINALITY ESTIMATION

ZENG ET AL. - 2024 - PRICE A PRETRAINED MODEL FOR CROSS-DATABASE

- Inspired by large language models (like BERT), PRICE learns "meta-knowledge" for CardEst
- It's universally applicable to any database, addressing transferability and high preparation costs
- Takes low-level, transferable features as input for a given SQL query Q on a database D



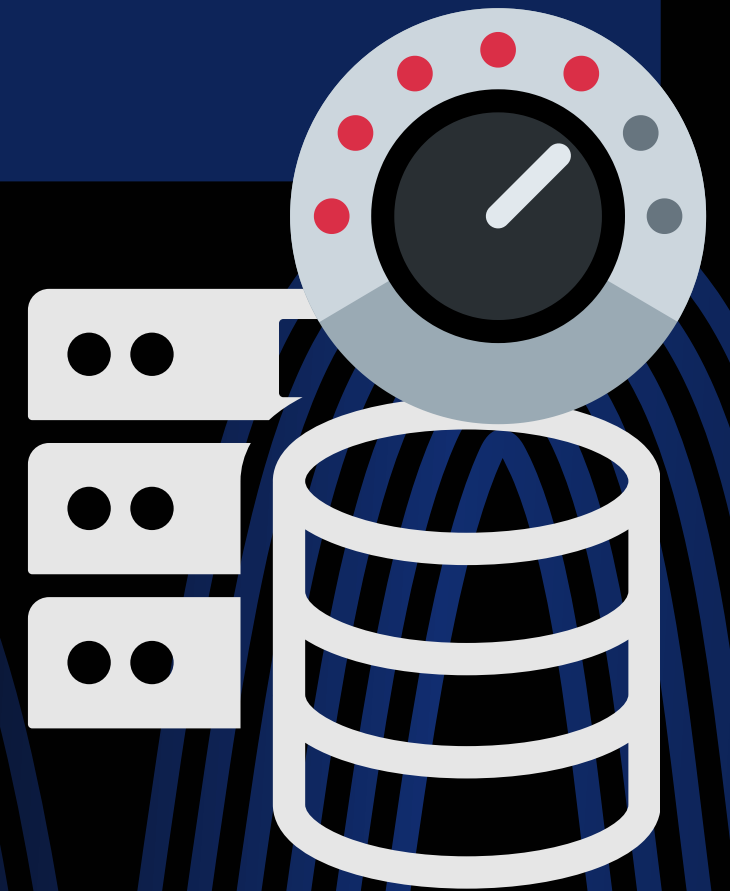
PRICE model architecture

KNOB TUNING

WHAT IS KNOB TUNING?

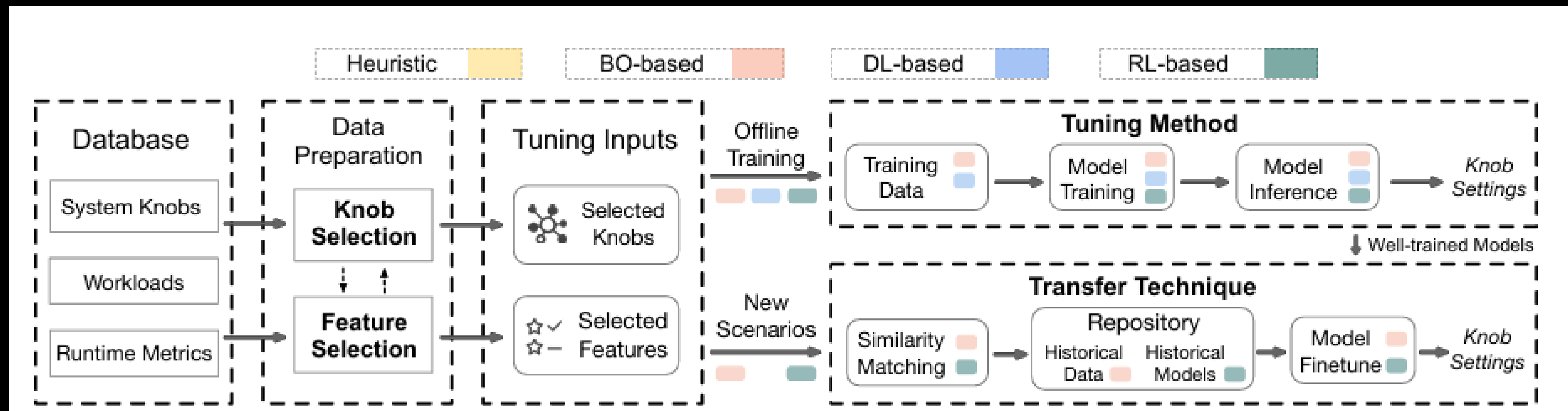
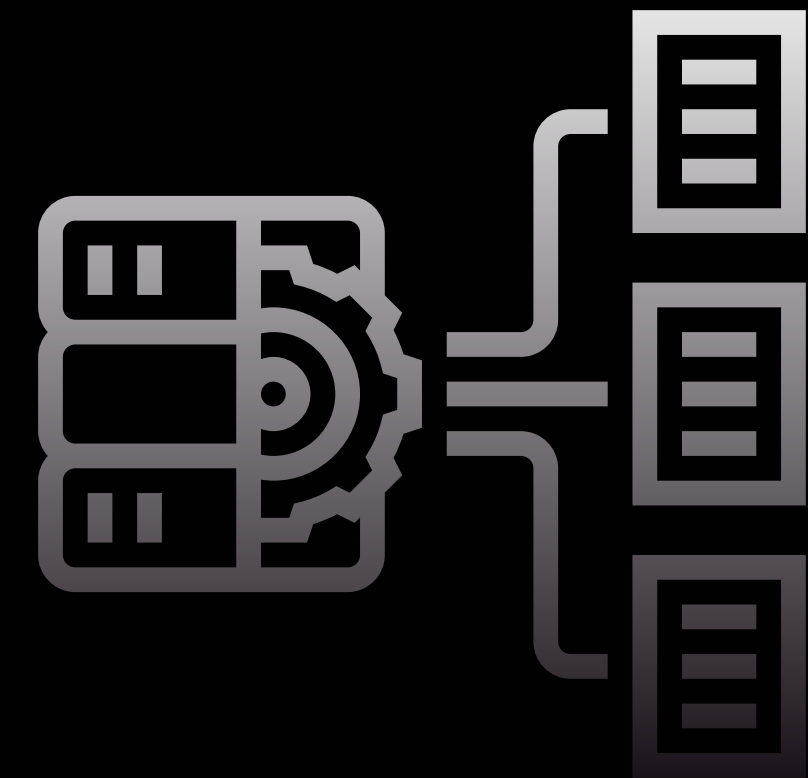
The process of configuring database system's internal parameters (knobs) to optimize query execution.

- **Objectives:**
 - Efficient resource utilization
 - Minimal latency
 - Maximal throughput for varying workloads.



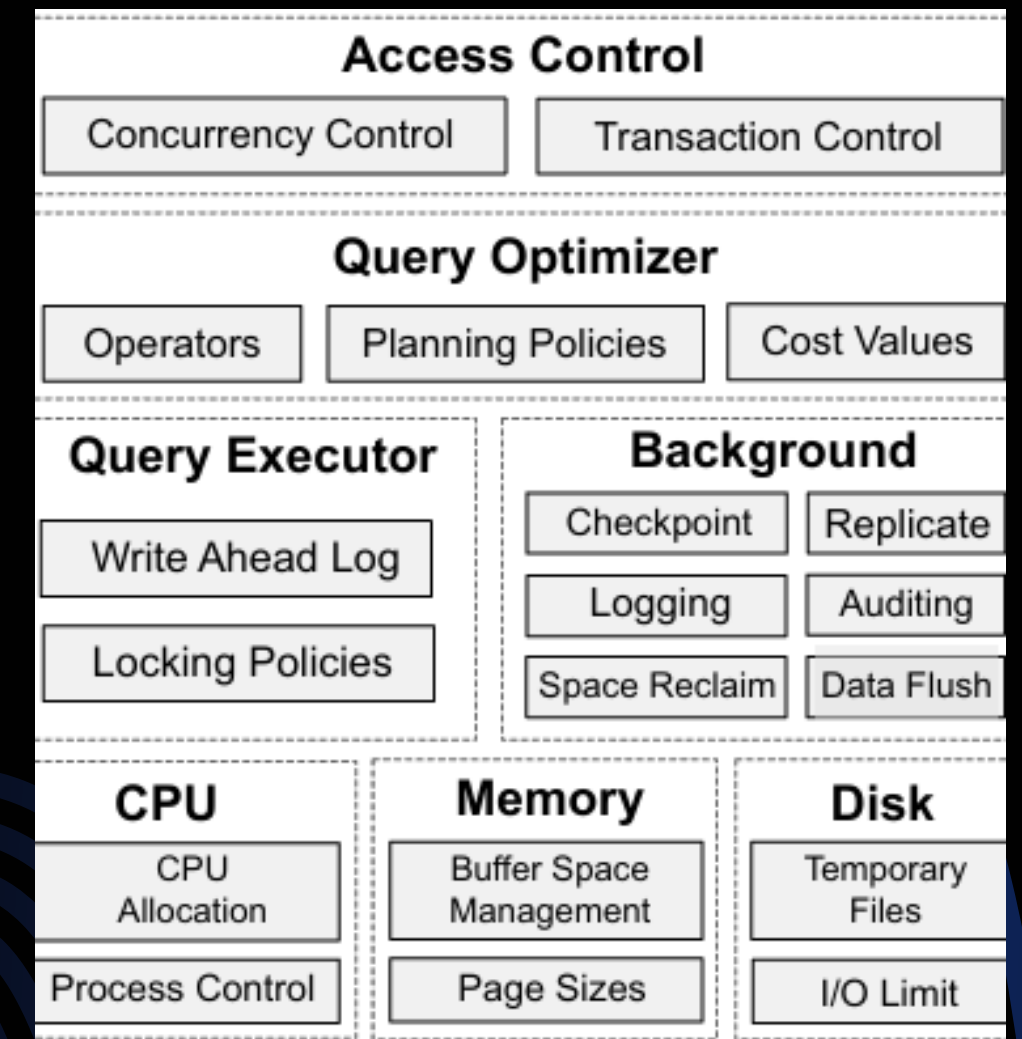
HOW DOES IT WORK?

1. Database Inputs
2. Data Preparation → Tuning Inputs
3. Tuning Methods → Optimal Knob Settings
4. Transfer Techniques



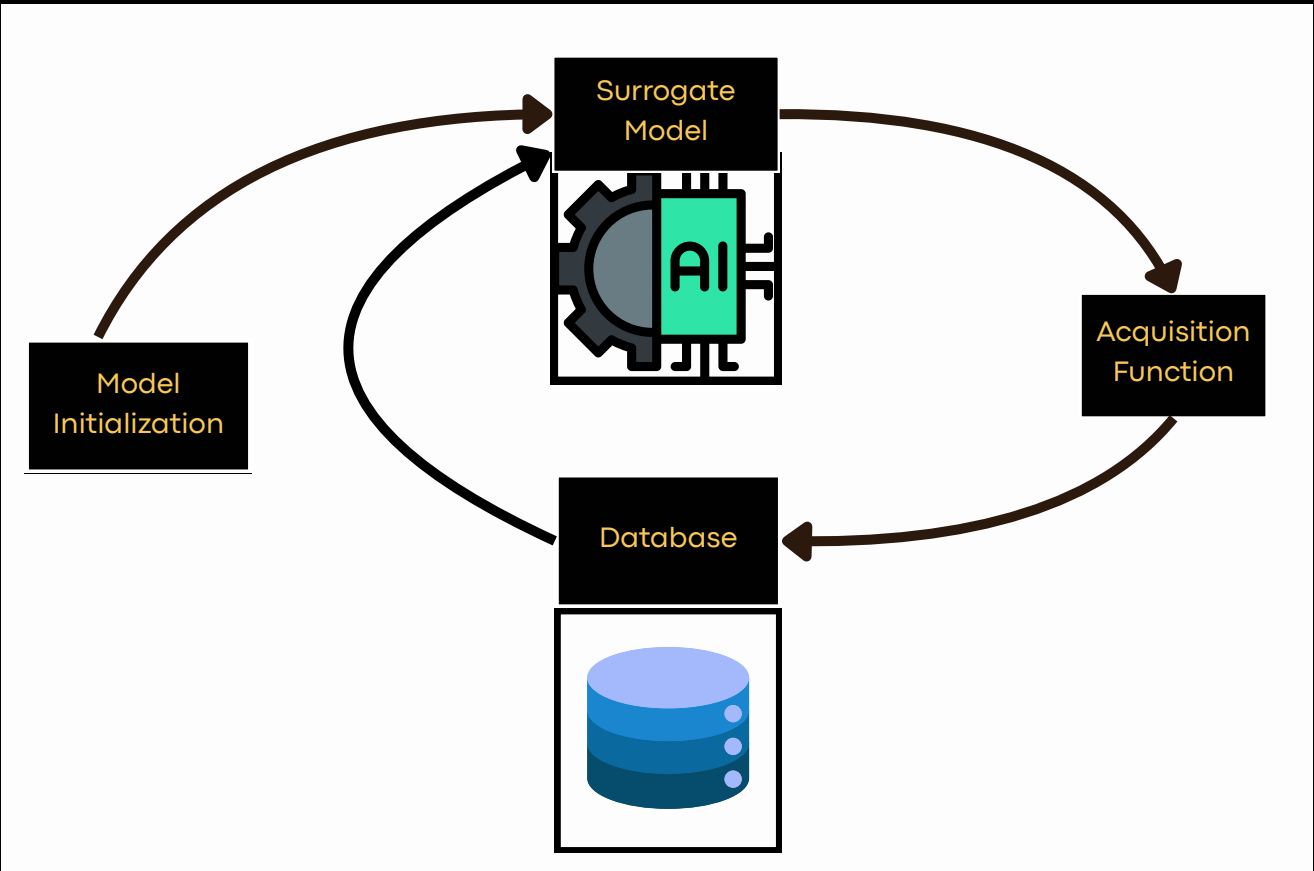
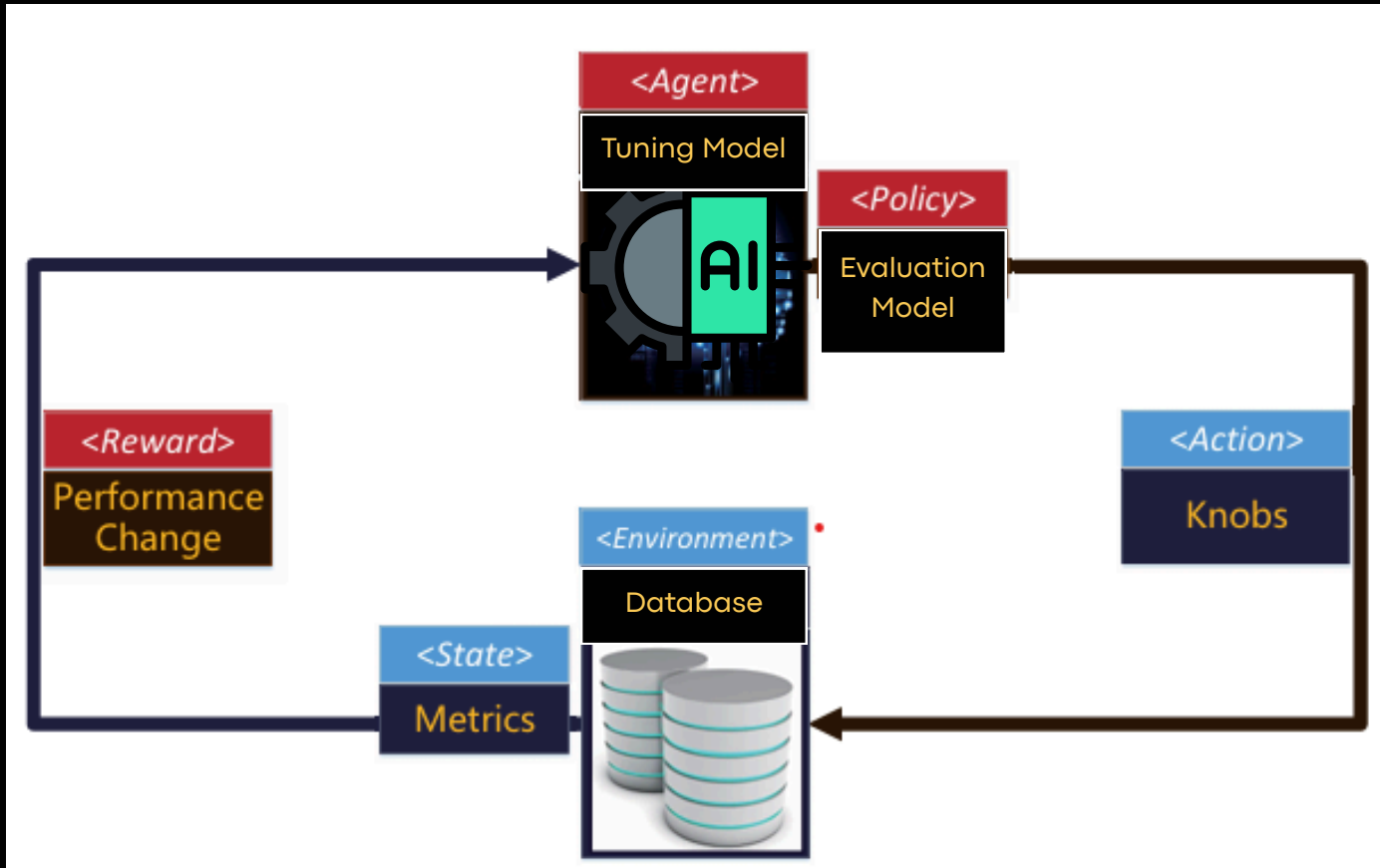
WHY KNOB TUNING IS A BOTTLENECK ?

- Suboptimal configurations due to High dimensionality
- Complex interdependencies
- Resource contention
- Dynamic workload impact



Most traditional databases rely on heuristic approaches or manual tuning, which fail to fully address these challenges.

AI-DRIVEN CONFIGURATION KNOB TUNING

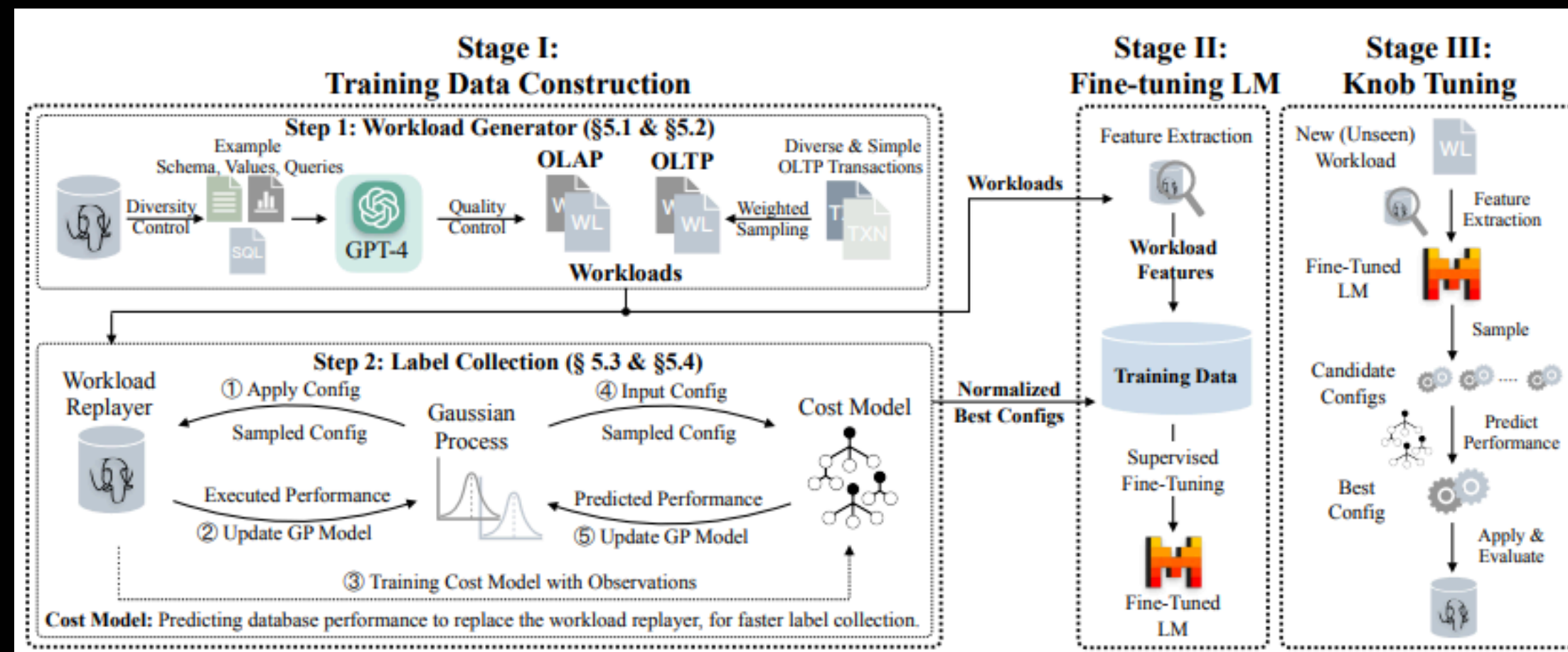
Category	Bayesian Optimization-based	Reinforcement Learning-based
Method	 The diagram illustrates the Bayesian Optimization-based method. It shows a cycle involving three main components: 'Model Initialization', 'Surrogate Model', and 'Acquisition Function'. 'Model Initialization' points to the 'Surrogate Model'. The 'Surrogate Model' (containing an 'AI' icon) points to the 'Acquisition Function'. The 'Acquisition Function' points to a 'Database' (represented by a cylinder icon). The 'Database' points back to the 'Surrogate Model', completing the loop.	 The diagram illustrates the Reinforcement Learning-based method. It shows a cycle involving several components: '<Agent>' (containing a 'Tuning Model' and an 'AI' icon), '<Policy>', 'Evaluation Model', '<Action>' (containing 'Knobs'), '<Environment>' (containing a 'Database'), '<State>' (containing 'Metrics'), and '<Reward>' (containing 'Performance Change'). The flow is as follows: '<Agent>' points to 'Evaluation Model'. 'Evaluation Model' points to '<Policy>'. '<Policy>' points to '<Action>'. '<Action>' points to '<Environment>'. '<Environment>' points to '<State>'. '<State>' points to '<Reward>'. '<Reward>' points back to '<Agent>', completing the loop.
Examples	iTuned, OtterTune, ReIM, ResTune, CGPTuner, OnlineTune, LlamaTune	CDBTune, QTune, UDO, WATuning, DB-BERT, HUNTER

AN OUTPERFORMING NOVEL KNOB TUNER

- Existing BO & RL methods aren't efficient due to iterative workload replays.
- Need a faster, end-to-end approach.

HUANG ET AL. - 2025 - E2ETUNE: END-TO-END KNOB TUNING VIA FINE-TUNED GENERATIVE LANGUAGE MODEL

- Inspired by skilled DBAs who use experience to choose good configs without long trial-and-error.
- Use a Fine-tuned Generative Language Model (e.g. Mistral7B-Instruct-v0.2) for sequence-to-sequence mapping.



E2ETune model architecture

LEARNED INDEXING

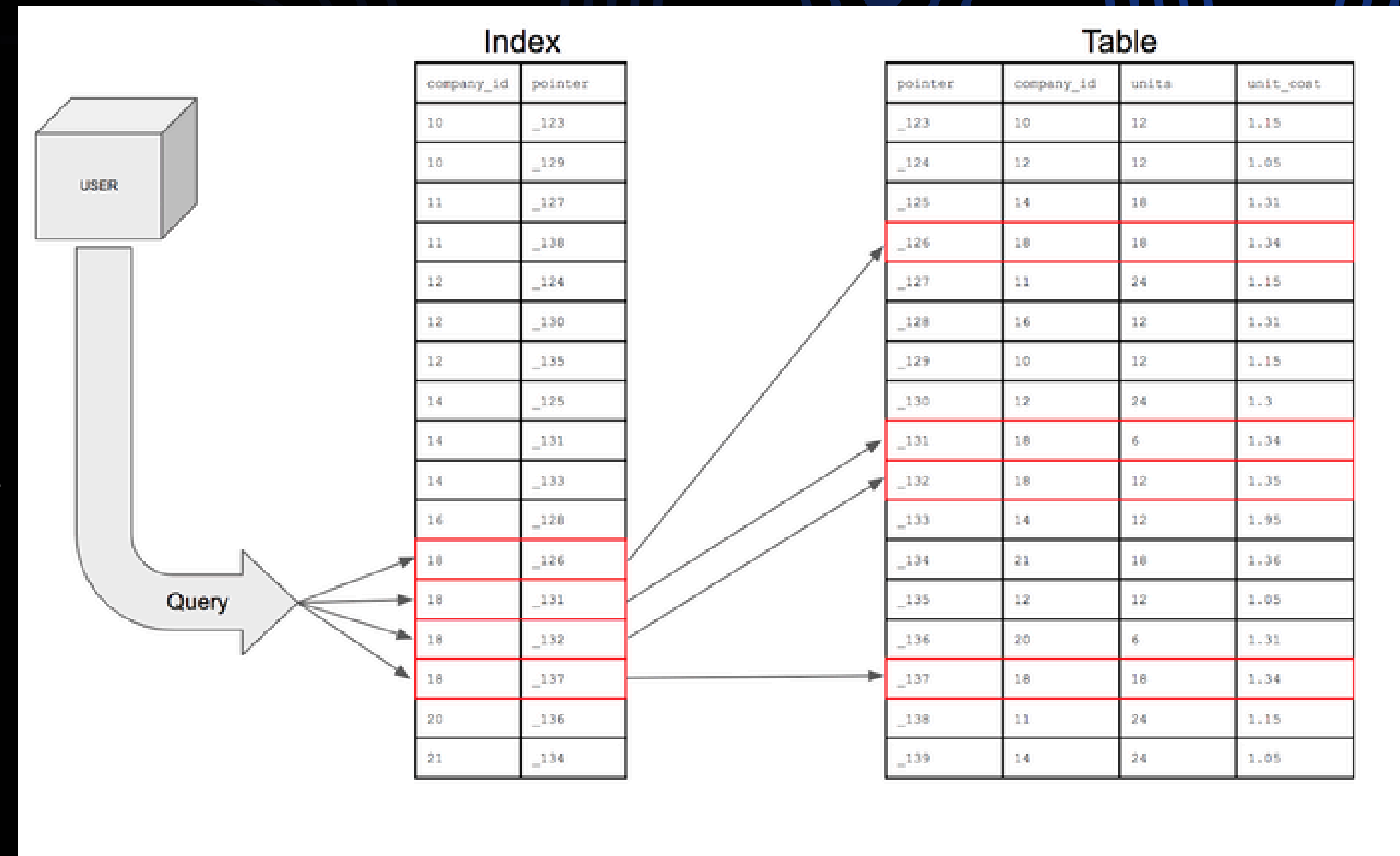
WHY DO WE NEED INDEXING?

Traditional Database Challenge:

- Finding data in large databases requires scanning entire tables (full table scans)
- Like reading every page of a book to find one topic
- As data grows to millions/billions of records, query performance degrades exponentially

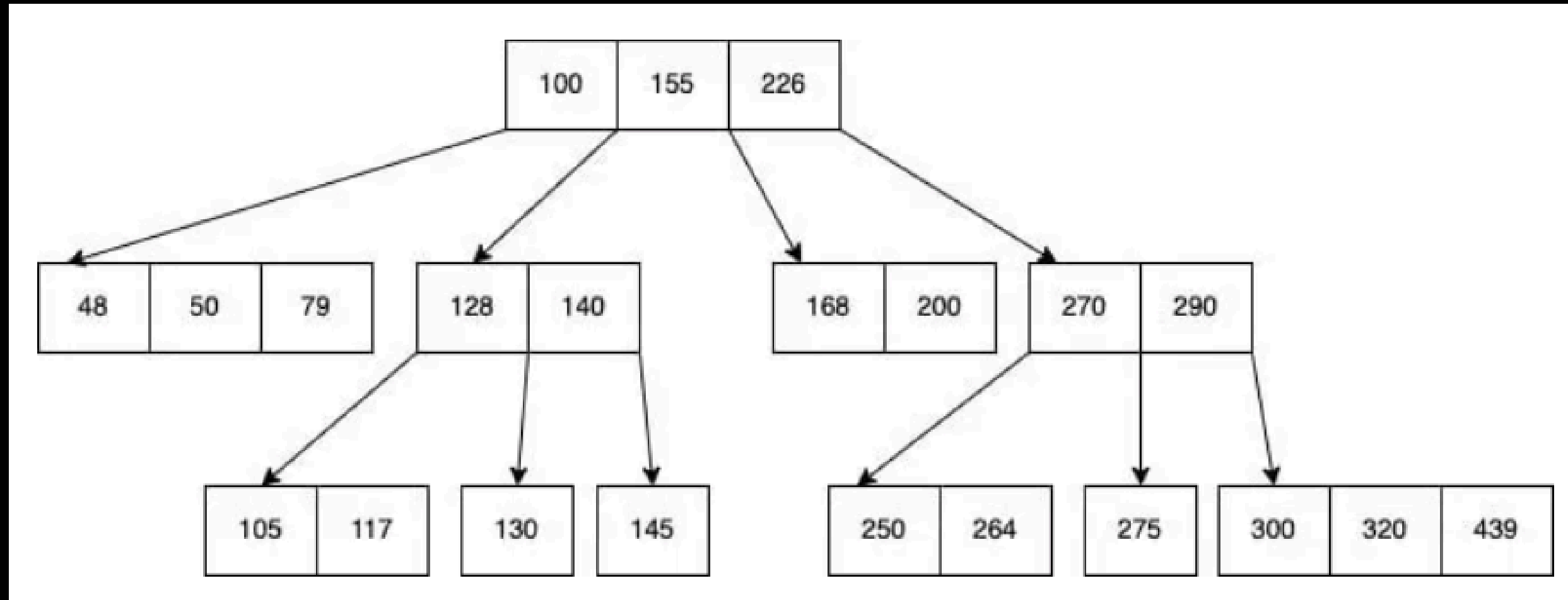
Core Objectives of Indexing:

- Speed: Reduce query time from seconds/minutes to milliseconds
- Efficiency: Navigate directly to relevant data without scanning
- Scalability: Maintain performance as datasets grow
- Resource Management: Optimize memory and storage usage



<https://www.atlassian.com/data/databases/how-does-indexing-work>

B+-TREES FOR INDEXING



https://miro.medium.com/v2/resize:fit:1100/format:webp/1*L-2cyVE_3Zwy9pTVFw8w_Q.jpeg

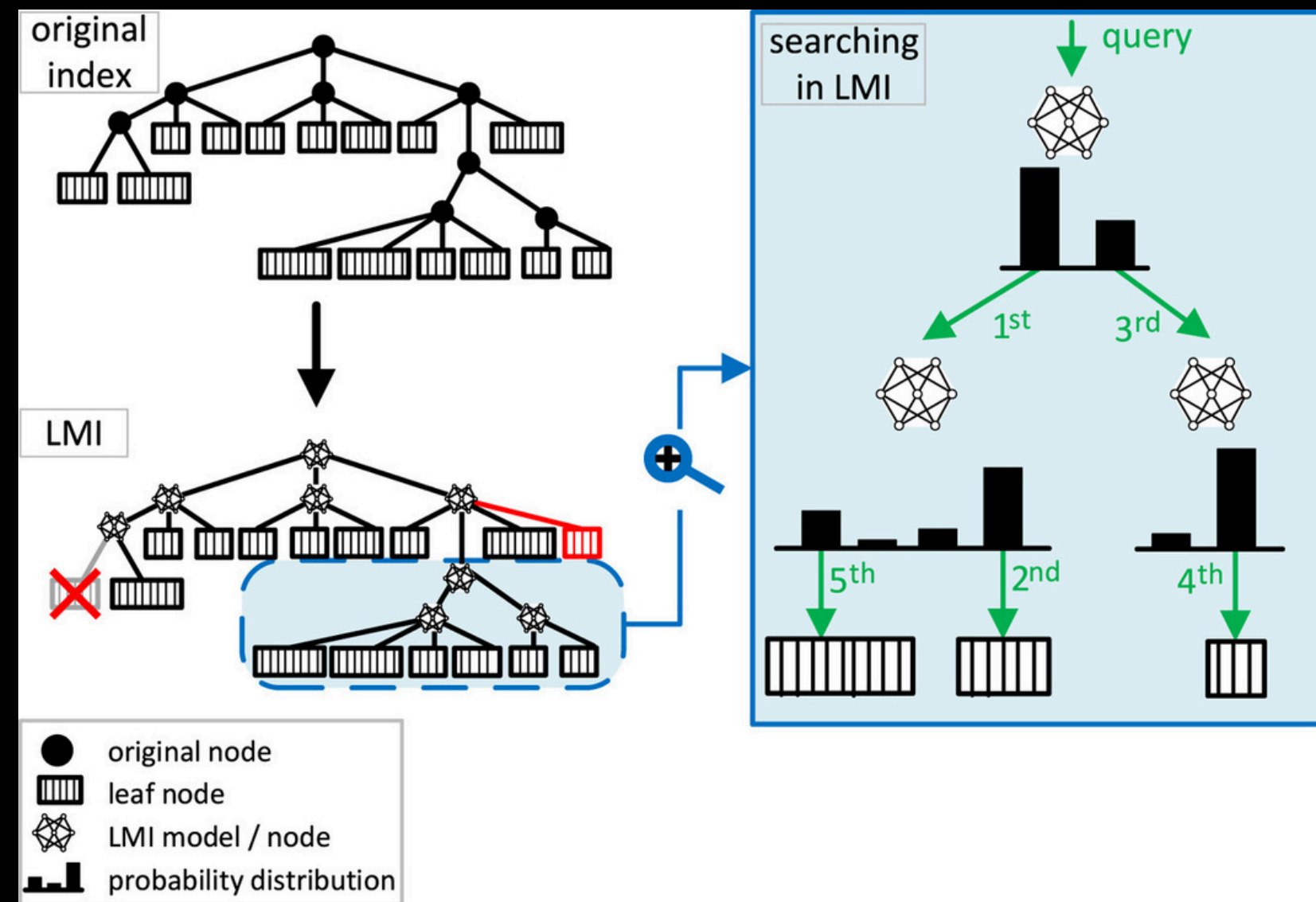
B+-Tree Problems

- **Pointer Chasing:** Multi-level tree traversal creates CPU cache misses
- **Memory Overhead:** Large memory footprint due to node structures and pointers
- **Update Costs:** Insertions/deletions require expensive structural reorganization
- **Distribution Blindness:** Cannot exploit data patterns or trends

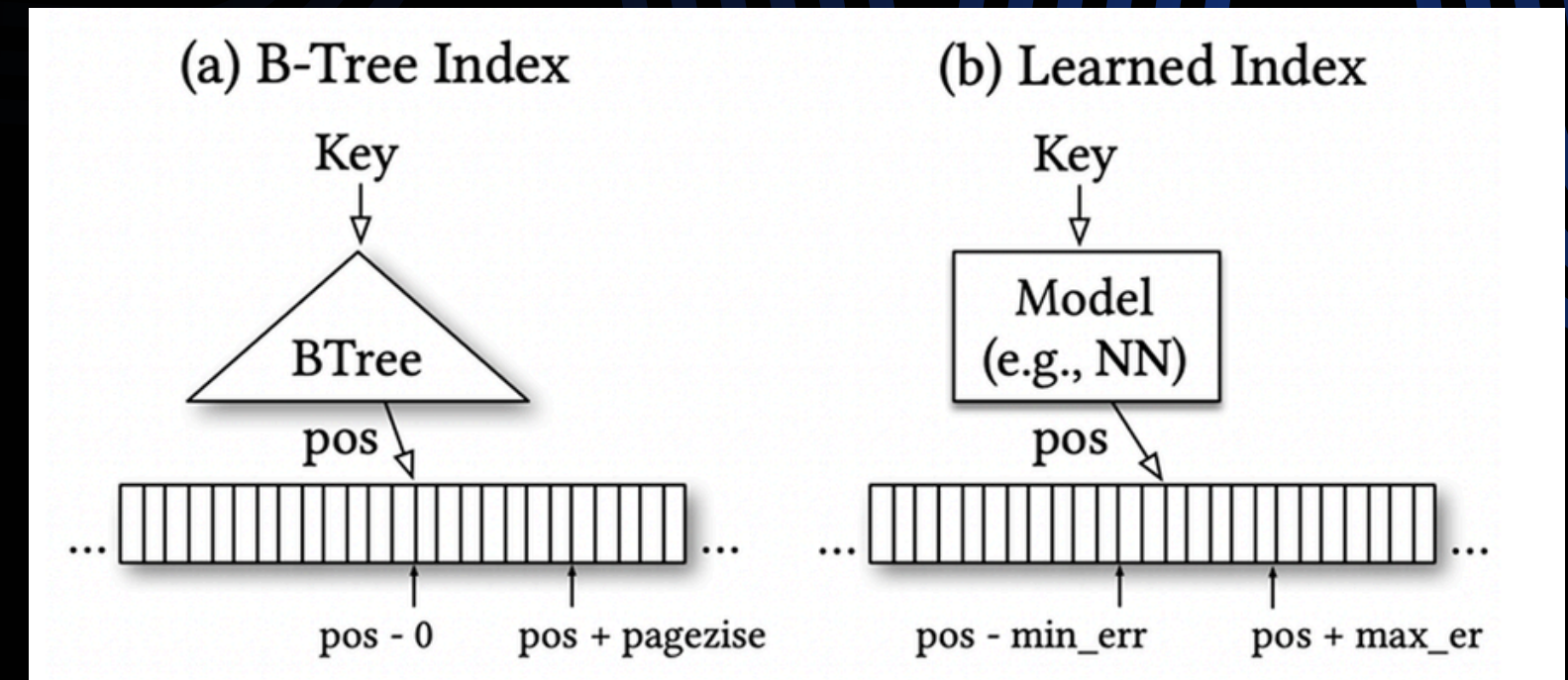
FROM B-TREES TO LEARNED INDEXING

What is a Learned Index?

Unlike B+-trees that map keys via tree traversal, learned indexes use machine-learning models (e.g., linear regression or neural nets) to predict a key's position.



Learned Metric Index architecture showing the model hierarchy and position prediction process



Comparison between traditional B-Tree (left) and Learned Index (right) structures

Core Paradigm Shift:

- Replace tree nodes with machine learning models (neural networks, linear regression)
- Learn the Cumulative Distribution Function (CDF) of sorted keys
- Predict key positions directly instead of navigating tree structures
- Models act as "learned functions" mapping keys to array positions

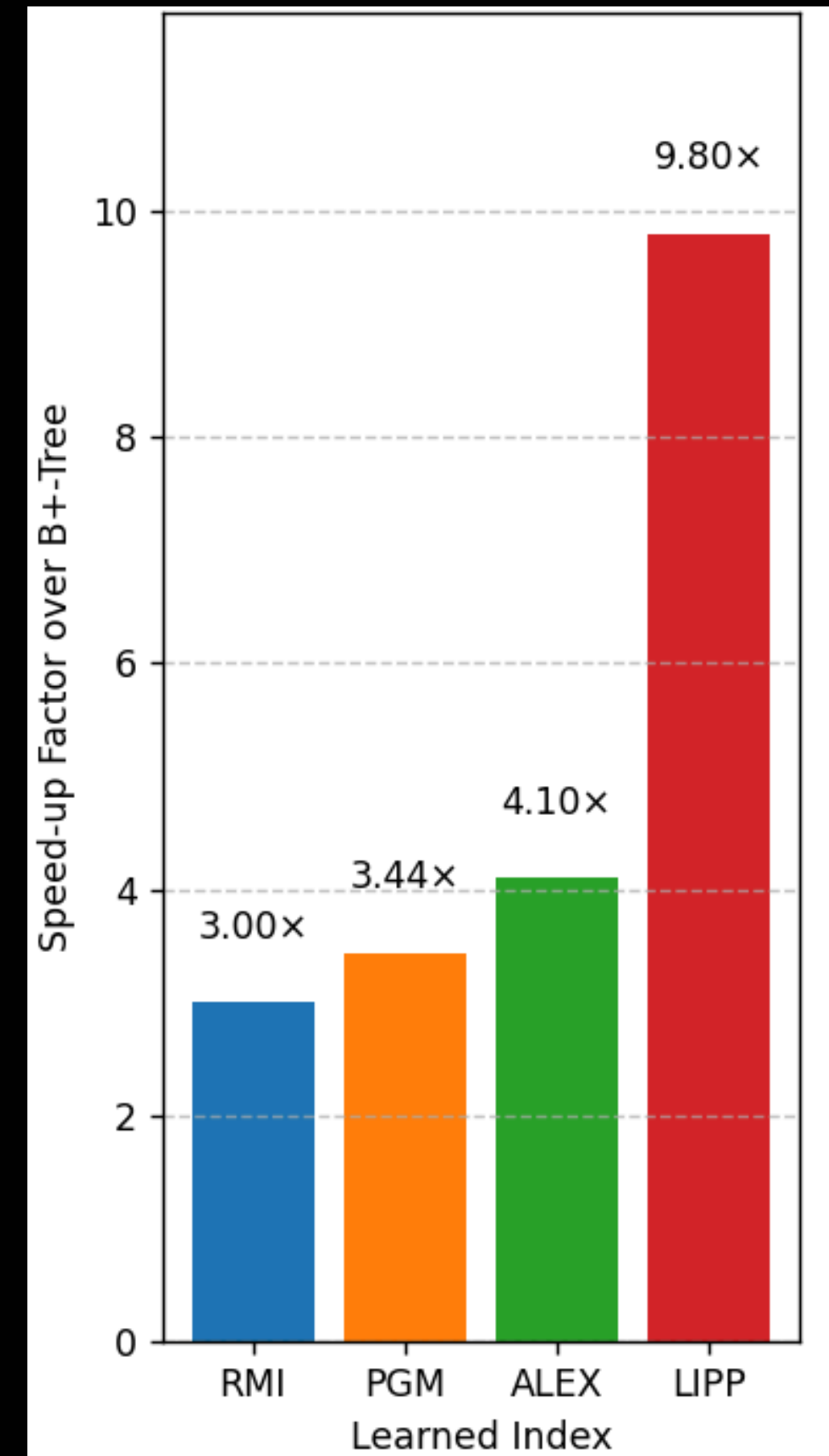
PERFORMANCE ADVANTAGES

Query Speed: Faster searches compared to B+-trees

Memory Efficiency: 2-4 orders of magnitude smaller space footprint

Large Static Datasets: Predictable, smooth data distributions

Cache Performance: Reduced branch mispredictions



Relative Performance Improvements of Learned Index Architectures compared to B+-trees

Proposed Method

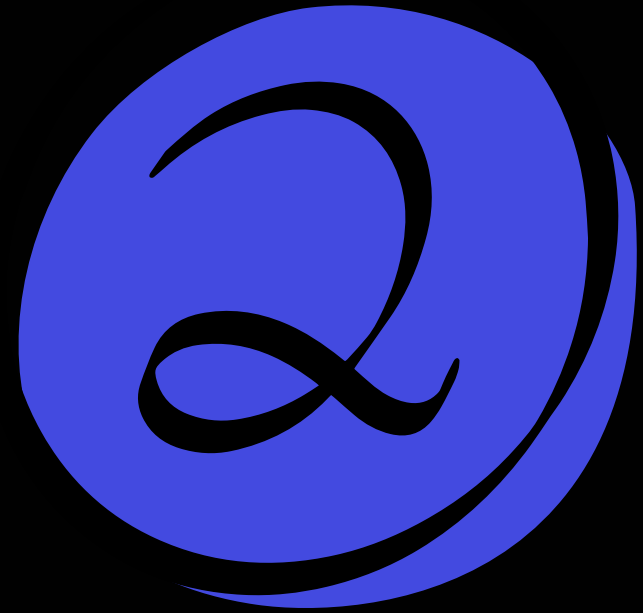


LITERATURE REVIEW AND BOTTLENECK ANALYSIS



Through our literature review we have identified bottlenecks and techniques for optimization, under the categories

- Query Optimization
- Knob Tuning
- Index Management



EVALUATING THE FEASIBILITY OF IMPLEMENTING LEARNED TECHNIQUES

Once candidate techniques are found under each category, we will explore the feasibility of implementing those techniques with resources available to us



PROTOTYPE IMPLEMENTATION

After a selective process, we will explore the feasibility of integrating chosen techniques to PostgreSQL

Due to PostgreSQL's extensible architecture we can extend or modify components to implement the chosen techniques.

Eg: Using Planner Hook to replace the standard planner

Called in query optimizer entry point.

If set, replaces standard planner. Consider inclusion of the standard planner to hook if this hook assuming just pre-process or post-process for builtin planner.

Inputs:

- *Query * parse* — parsed query text.
- *const char * query_string* — original query text.
- *int cursorOptions*
- *ParamListInfo boundParams*



PERFORMANCE EVALUATION

Conduct a rigorous experimental evaluation to quantify the performance gains of the implemented optimizations compared to the unmodified database.

- Defining key performance metrics - Aspects such as query latency, robustness under workload drift, and inference overhead will be evaluated.
- Benchmarks - TPC-H, JOB, IMDB workloads
- Identify any trade-offs

Feasibility & Scope

Strong Foundation

- A key enabler of feasibility is the availability of **PostgreSQL**, a widely used open-source database system, which provides a mature codebase and extension APIs that we can build upon.

Technical Risks and Mitigation

- Direct modification of PostgreSQL internals may prove complex. To address this, we will adopt a two-phase integration strategy:

What makes the project feasible?

1



Starting with PilotScope-based middleware integration for rapid experimentation

2



Selective PostgreSQL source modifications where feasible

Resource Availability

- Publicly available standard DBMS benchmarks and workloads.
JOB, IMDB
- Models can be trained with low resources as the techniques are proposing lightweight models.

<https://developer.imdb.com/non-commercial-datasets/>

<https://github.com/andreaskipf/learnedcardinalities/blob/master/workloads/job-light.sql>

Scope

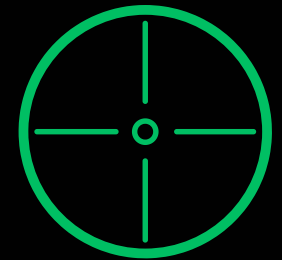
Addressing three major performance bottlenecks in relational databases

Query Optimization

Indexing

Configuration Tuning

Based on our literature review and comparative evaluations we will limit prototype integration to 1–2 high-impact techniques.



IN SCOPE

The project does not aim to deliver a production-ready DBMS fork. Instead, the focus is on proof-of-concept integration and systematic evaluation. Packaging, distribution, and long-term maintenance of a new database system are beyond the scope of this project.

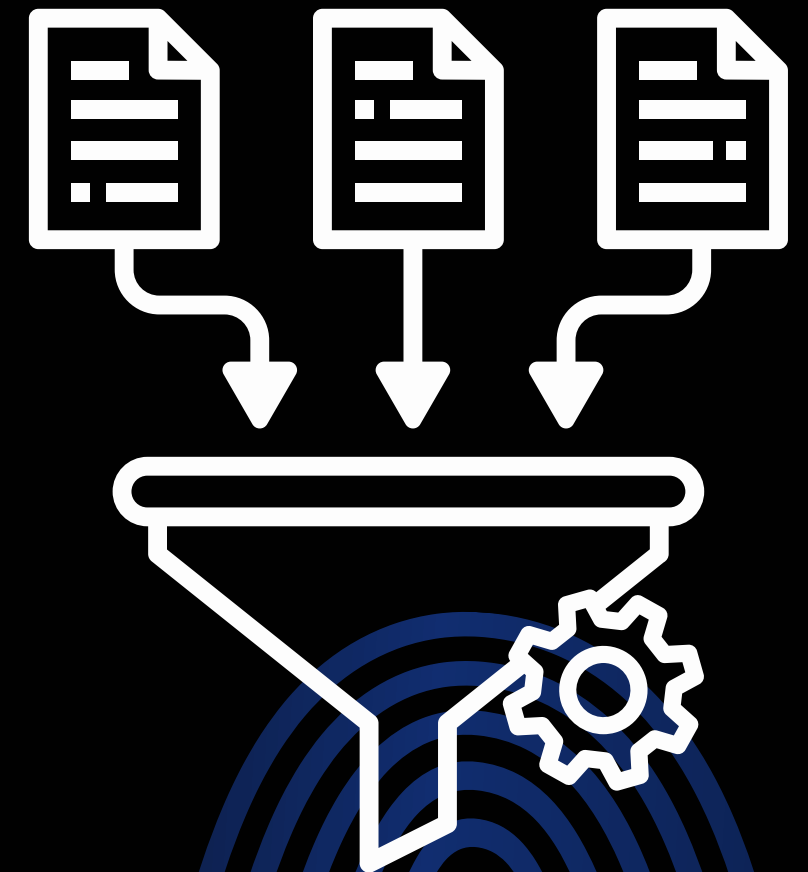


OUT OF SCOPE

Deliverables

Our Target Outcomes

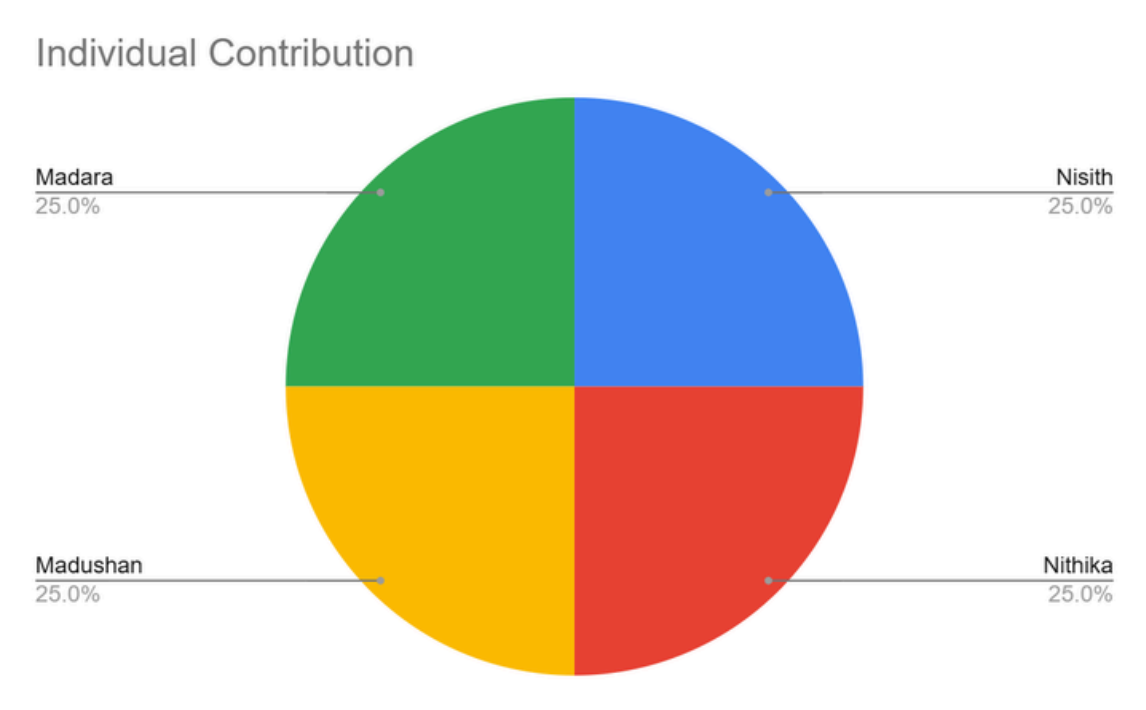
- **Survey:** Review of learned database optimization techniques, highlighting strengths, weaknesses, and research gaps.
- **Prototype:** PostgreSQL integration with selected learned components.
- **Recommendations:** Practical guidelines for when and how to adopt learned optimizations.
- **Final Report:** Research paper consolidating findings and contributions.



Timeline & Work Distribution

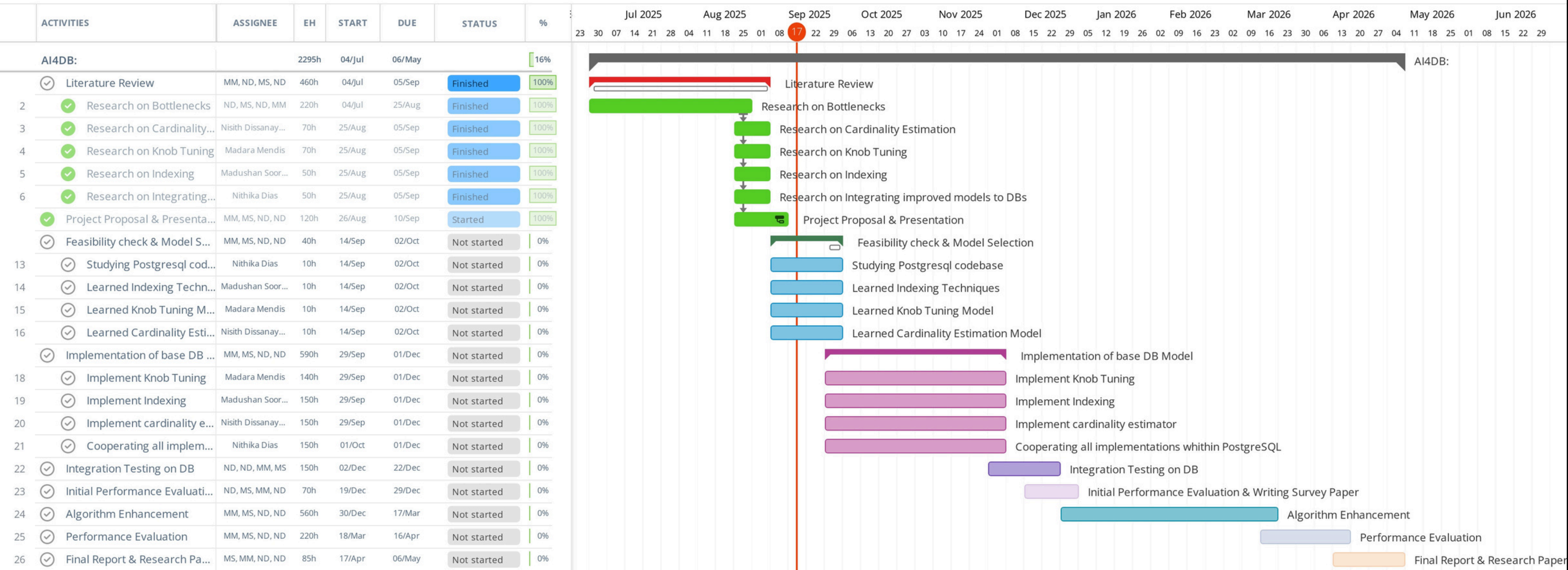
Work Distribution

Member	Key, Responsible Areas
210125B - Nithika Dias	Integration testing into PostgreSQL
210144G - Nisith Dissanayake	Query Optimization
210626L - Madushan Suriyabandara	Index Management
210386A - Madara Mendis	Knob Tuning



Name	Start	Finish	Duration(days)	% Complete	Nisith	Nithika	Madushan	Madara	Total Hours
Literature Review	04/07/2025	05/09/2025	46	50	130	100	100	130	460
Project Proposal & Presentation	21/08/2025	05/09/2025	12	100	20	40	40	20	120
Feasibility Check & Model Selection	05/09/2025	10/09/2025	4	0	10	10	10	10	30
Implementation of Base DB Model	10/09/2025	01/12/2025	59	0	150	150	150	140	590
Integration Testing on DB	02/12/2025	22/12/2025	15	0	37	38	38	37	150
Initial Performance Evaluation	19/12/2025	29/12/2025	7	0	18	17	17	18	70
Algorithm Enhancement	30/12/2025	17/03/2026	56	0	140	140	130	150	560
Performance Evaluation	18/03/2026	16/04/2026	22	0	55	55	55	55	220
Final Report & Research Paper	17/04/2026	06/05/2026	14	0	25	35	45	25	140
			235		585	585	585	585	2340

Timeline



References

[1]
P. Li, W. Wei, R. Zhu, B. Ding, J. Zhou, and H. Lu, “ALECE: An Attention-based Learned Cardinality Estimator for SPJ Queries on Dynamic Workloads (Extended),” Proc. VLDB Endow., vol. 17, no. 2, pp. 197–210, Oct. 2023, doi: [10.14778/3626292.3626302](https://doi.org/10.14778/3626292.3626302).

[2]
B. Hilprecht, A. Schmidt, M. Kulesa, A. Molina, K. Kersting, and C. Binnig, “DeepDB: learn from data, not from queries!,” Proc. VLDB Endow., vol. 13, no. 7, pp. 992–1005, Mar. 2020, doi: [10.14778/3384345.3384349](https://doi.org/10.14778/3384345.3384349).

[3]
Z. Yang et al., “NeuroCard: one cardinality estimator for all tables,” Proc. VLDB Endow., vol. 14, no. 1, pp. 61–73, Sept. 2020, doi: [10.14778/3421424.3421432](https://doi.org/10.14778/3421424.3421432).

[4]
V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, “How good are query optimizers, really?,” Proc. VLDB Endow., vol. 9, no. 3, Nov. 2015, doi: [10.14778/2850583.2850594](https://doi.org/10.14778/2850583.2850594).

[5]
T. Zeng et al., “PRICE: A Pretrained Model for Cross-Database Cardinality Estimation,” June 03, 2024, arXiv: arXiv:2406.01027. doi: [10.48550/arXiv.2406.01027](https://doi.org/10.48550/arXiv.2406.01027).

[6]
A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper, “Learned Cardinalities: Estimating Correlated Joins with Deep Learning,” Dec. 18, 2018, arXiv: arXiv:1809.00677. doi: [10.48550/arXiv.1809.00677](https://doi.org/10.48550/arXiv.1809.00677).

[7]
X. Huang, H. Li, J. Zhang, X. Zhao, Z. Yao, Y. Li, T. Zhang, J. Chen, H. Chen, and C. Li, “E2ETune: End-to-End Knob Tuning via Fine-tuned Generative Language Model,” Proc. VLDB Endowment, vol. 18, no. 5, 2025. doi: [10.48550/arXiv.2404.11581](https://doi.org/10.48550/arXiv.2404.11581)

[8]
X. Zhao, X. Zhou, and G. Li, “Automatic Database Knob Tuning: A Survey,” IEEE Transactions on Knowledge and Data Engineering, vol. 35, no. 12, pp. 12470–12490, Dec. 2023. doi: [10.1109/TKDE.2023.3266893](https://doi.org/10.1109/TKDE.2023.3266893)

Thank You